Upgrading a Self-Managed Kubernetes Cluster in Production

A **self-managed Kubernetes upgrade** requires a **careful, step-by-step approach** to **prevent downtime, service disruptions, and breaking changes**. Below is a **detailed, production-grade guide** to safely **upgrade your Kubernetes cluster**

Step 1: Plan the Upgrade

✅ 1.1 Check Kubernetes Version Skew Policy

- Kubernetes allows **only one minor version difference** between:
  - **Control Plane Nodes & Worker Nodes**
  - **Kubelet & API Server**
- This means **you CANNOT skip versions** and must upgrade from **1.29 → 1.30** step by step.

✅ 1.2 Review Kubernetes Deprecation & Breaking Changes

- Read the **Kubernetes Release Notes**:
  - 🚨 **Check API Deprecations** using:

```
kubectl get apiservices
kubectl get apiversions
```

  - 🚨 **Check for removed features**
  - 🚨 **Ensure compatibility with storage, networking, and RBAC**

✅ 1.3 Verify Cluster Add-ons, Plugins & CSI Drivers

- Check **CNI Plugins** (Calico, Flannel, Cilium, etc.) support the version you are upgrading to.
- Verify **Ingress Controllers (Nginx, Traefik, Istio, etc.)** support the version you are upgrading to.
  Ensure **Storage Drivers (CSI, EBS, Ceph, etc.)** are compatible.

Step 2: Backup & Prepare for Rollback

✅ 2.1 Take Full Cluster Backup

Before upgrading, **backup all critical data**:

Backup ETCD (Control Plane State)

```
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-snapshot.db --endpoints=https://127.0.0.1:2379
```

Backup Kubernetes Resources

```
kubectl get all --all-namespaces -o yaml > backup-all-resources.yaml
kubectl get configmap -n kube-system -o yaml > backup-configmaps.yaml
kubectl get secrets --all-namespaces -o yaml > backup-secrets.yaml
```

Use Velero for Cloud-Based Backups

```
velero backup create cluster-backup --include-cluster-resources=true
```

✅ 2.2 Set Up Rollback Plan

- If **upgrade fails**, be ready to **restore ETCD snapshot**:

```
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-snapshot.db
```

- **Test rollback in a staging environment before applying to production.**

Step 3: Upgrade the Kubernetes Cluster

3.1 Upgrade Control Plane (Master Nodes)

👉 Step 1: Drain & Cordon One Control Plane Node

```
kubectl drain <master-node> --ignore-daemonsets --delete-emptydir-data
kubectl cordon <master-node>
```

👉 Step 2: Upgrade `kubeadm` to v1.30

```
apt update && apt install -y kubeadm=1.30.0-00
```

- **Verify upgrade plan:**

```
kubeadm upgrade plan
```

👉 Step 3: Apply the Upgrade

```
kubeadm upgrade apply v1.30.0
```

- ✅ **Automatically upgrades:**
  - `kube-apiserver`
  - `kube-controller-manager`
  - `kube-scheduler`
  - `etcd`

👉 Step 4: Upgrade `kubelet` and `kubectl`

```
apt install -y kubelet=1.30.0-00 kubectl=1.30.0-00
systemctl restart kubelet
```

👉 Step 5: Uncordon the Control Plane Node

```
kubectl uncordon <master-node>
```

👉 Step 6: Repeat for All Master Nodes

- **Upgrade control plane nodes one by one** (if running HA).

3.2 Upgrade Worker Nodes

👉 Step 1: Drain & Cordon a Worker Node

```
kubectl drain <worker-node> --ignore-daemonsets --delete-emptydir-data
kubectl cordon <worker-node>
```

👉 Step 2: Upgrade `kubeadm`

```
apt update && apt install -y kubeadm=1.30.0-00
```

```
kubeadm upgrade node
```

👉 Step 3: Upgrade `kubelet` and `kubectl`

```
apt install -y kubelet=1.30.0-00 kubectl=1.30.0-00
systemctl restart kubelet
```

👉 Step 4: Uncordon the Worker Node

```
kubectl uncordon <worker-node>
```

👉 **Step 5: Repeat for All Worker Nodes**

- **One node at a time** to **avoid cluster downtime**.

Step 4: Verify the Upgrade

✅ 4.1 Check Kubernetes Version

```
kubectl version --short
```

✅ 4.2 Check Node & Pod Health

```
kubectl get nodes
kubectl get pods --all-namespaces
```

✅ 4.3 Verify Cluster Functionality

- Test **DNS resolution**:

```
kubectl run test-pod --image=busybox --rm -it -- nslookup kubernetes.default
```

- Check **Kube API availability**:

```
kubectl get apiservices
```

- Ensure **all workloads are running as expected**.

Step 5: Post-Upgrade Best Practices

✅ 5.1 Enable Kubernetes API Deprecation Warnings

- Use `kubectl convert` to **detect outdated API versions**.

```
kubectl convert -f old-manifest.yaml --output-version v1
```

✅ 5.2 Update Helm Charts & CRDs

- If using Helm:

```
helm repo update
helm upgrade --install my-release my-chart
```

- Update Custom Resource Definitions (CRDs) if needed.

✅ 5.3 Reapply Security Policies & Networking Configurations

- Ensure **Network Policies** and **RBAC policies** are intact.

```
kubectl get networkpolicies --all-namespaces
kubectl get roles,rolebindings,clusterroles,clusterrolebindings --all-namespaces
```

Step 6: Implement Automation for Future Upgrades

✅ 6.2 Implement Kubernetes Lifecycle Management

- Use **Cluster API (CAPI)** to automate upgrades.
- Use **Ansible playbooks or Terraform** for cluster updates.

Final Summary

Step	Action
1	Plan the upgrade, check deprecations
2	Backup ETCD, cluster configs, and PVs
3	Upgrade Control Plane **(one node at a time)**
4	Upgrade Worker Nodes **(one node at a time)**
5	Verify cluster health, workloads, networking
6	Automate upgrades using GitOps (ArgoCD, FluxCD)

---

more clarity regarding few concepts

If using Helm charts

With respect to Helm

- Make sure to upgrade the helm repos first
- Take the backup of helm charts, values files to other directory
- Perform the upgrade

Upgrade Helm Charts Before Upgrading Kubernetes

Before upgrading Kubernetes, make sure **Helm charts are updated to a version that supports Kubernetes v1.30**.

👉 **Step 1: Update the Helm repository**

```
helm repo update
```

👉 **Step 2: Check available chart versions**

```
helm search repo <chart-name>
```

👉 **Step 3: Upgrade the Helm chart**

```
helm upgrade --install <release-name> <chart-name> -n <namespace> --values backup-helm-values.yaml
```

🚨 **If the chart is outdated and does not support Kubernetes v1.30, DO NOT proceed with the upgrade.** 📌 **Upgrade Kubernetes only after ensuring all Helm charts are updated.**

Backup Helm Releases

Since Helm is used to manage applications, **always take a backup** before upgrading Kubernetes.

👉 **Backup Helm manifests (CRDs, deployments, services, etc.)**

```
helm get manifest <release-name> -n <namespace> > backup-helm-manifest.yaml
```

👉 **Backup Helm values**

```
helm get values <release-name> -n <namespace> > backup-helm-values.yaml
```

👉 **Backup entire Helm release (if using Helm v3)**

```
helm pull <chart-name> --version <chart-version> --untar
```

Post-Upgrade Helm Validation

After upgrading Kubernetes: 👉 **Step 1: Check if all Helm releases are running properly**

```
helm list --all-namespaces
```

👉 **Step 2: Verify Helm applications are healthy**

```
kubectl get pods --all-namespaces
kubectl get deployments --all-namespaces
kubectl get svc --all-namespaces
```

👉 **Step 3: Restart Helm-managed pods (if needed)**

```
kubectl rollout restart deployment <deployment-name> -n <namespace>
```

👉 **Step 4: If any Helm-managed application is broken, rollback using:**

```
helm rollback <release-name> <revision>
```

Check the Helm Chart's Kubernetes Compatibility

Every Helm chart has a `Chart.yaml` file that specifies the **Kubernetes versions it supports**.

Method 1: Check Helm Chart Kubernetes Version in `Chart.yaml`

1. Find the Helm chart directory: ""helm show chart <chart-name>""
2. Look for the **`kubeVersion`** field:

```
apiVersion: v2
name: my-chart
version: 1.2.3
kubeVersion: ">=1.25.0 <1.30.0"
```

- If the **`kubeVersion` does NOT include v1.30**, the chart might be **incompatible with Kubernetes v1.30**.
- If `kubeVersion: ">=1.30"` is **not listed**, you need an **updated version of the Helm chart**.

Understanding the `etcdctl` Backup Command in Detail

```
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-snapshot.db --endpoints=https://127.0.0.1:2379
```

This command **creates a backup (snapshot) of the etcd database**, which stores all Kubernetes cluster data, including:

- Cluster state
- API objects
- Configuration settings

This backup is **critical** for disaster recovery if something goes wrong during a Kubernetes upgrade.

Breaking Down the Command

Command Part	Explanation
`ETCDCTL_API=3`	Specifies that we are using **etcd API v3** (newer and recommended for Kubernetes).
`etcdctl snapshot save`	The `etcdctl` command to **save a backup (snapshot) of the etcd database**.
`/backup/etcd-snapshot.db`	**File path where the backup is stored**. In this case, it is saved as `etcd-snapshot.db` inside the `/backup/` directory.
`--endpoints=` https://127.0.0.1:2379	

	Defines the etcd server endpoint(s) where the cluster stores data. 127.0.0.1:2379 means the etcd server is running **locally on this node** at port `2379` (default etcd port).

Where is the Backup Stored?

- The backup is saved at: "/backup/etcd-snapshot.db"
- This means the snapshot is stored **locally on the node where the command is executed**.
- If needed, **you should copy it to a remote backup location (e.g., S3, NFS, external storage)** "aws s3 cp /backup/etcd-snapshot.db s3://my-backup-bucket/"

Why is `--endpoints` Used?

- `--endpoints=` **https://127.0.0.1:2379** specifies the etcd server **URL where the cluster data is stored**.
- In a **multi-node (HA) setup**, etcd runs on multiple nodes, so you might need to provide multiple endpoints: "--endpoints=https://etcd-1:2379,https://etcd-2:2379,https://etcd-3:2379"

This ensures you get a **consistent backup** from the etcd cluster.

Can I Delete the Backup After a Successful Upgrade?

Yes, **once the Kubernetes upgrade is successful and stable**, you can safely delete the backup.

👉 **You only need the backup if the upgrade fails**, allowing you to **restore the etcd database and recover the cluster**.

✅ **Safe steps to delete the backup:**

1. **Verify cluster health:**

```
kubectl get nodes
kubectl get pods --all-namespaces
```

1. **Ensure API server & etcd are stable:**

```
kubectl get apiservices
kubectl get componentstatus
```

1. **If everything is stable, delete the backup: "rm -f /backup/etcd-snapshot.db"**

Understanding the kubectl drain and kubectl cordon Commands in Detail

When upgrading a **self-managed Kubernetes cluster**, you need to **safely remove workloads from nodes** to prevent disruptions.The two main commands used for this are:

```
kubectl drain <master-node> --ignore-daemonsets --delete-emptydir-data
kubectl cordon <master-node>
```

These commands help **prepare a node for maintenance (such as upgrading, rebooting, or replacing it)** by **safely evicting workloads and preventing new ones from scheduling on it**.

What Does kubectl drain Do?

```
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data
```

Meaning:

- The kubectl drain **command safely evicts all running pods** from the specified node before performing maintenance.
- It ensures **no new workloads** get scheduled on the node **until it is uncordoned**.

What Happens When You Run kubectl drain?

1️⃣ **Marks the Node as Unschedulable:**

- New pods **will NOT be scheduled** on this node.
- Equivalent to running kubectl cordon.

2️⃣ **Evicts All Non-DaemonSet Pods:**

- All regular **Deployment, StatefulSet, and ReplicaSet** pods are **moved to other healthy nodes**.
- Any **running pods are terminated** and recreated elsewhere in the cluster.

3️⃣ **Handles Pod Disruptions Gracefully:**

- **Respects Pod Disruption Budgets (PDBs):**
  - If a pod **belongs to a PDB** that **limits disruptions**, drain waits **until the budget allows eviction**.
- If a pod **has a `local-storage` volume (`emptyDir`)**, drain **fails** unless `--delete-emptydir-data` is used.

Why Do We Need kubectl drain?

1️⃣ **Prevents Service Disruptions:**

- Moves workloads to **other healthy nodes** to keep the application running.

2️⃣ **Ensures Clean Upgrade/Maintenance:**

- Makes sure **no application data is lost or left behind**.

3️⃣ **Prevents Resource Starvation:**

- Ensures the node is **empty** before shutting it down.

Explanation of Drain Flags

Flag	Purpose
`--ignore-daemonsets`	Skips eviction of **DaemonSet** pods (like monitoring, logging, networking agents).
`--delete-emptydir-data`	Deletes **emptyDir volumes**, which are temporary storage directories that disappear when a pod is deleted.
`--force`	Forcefully drains the node even if certain errors occur.

What Does kubectl cordon Do?

```
kubectl cordon <node-name>
```

Meaning:

- **Cordon marks the node as unschedulable, preventing new pods from being scheduled on it**.
- However, **existing pods continue running**.

What Happens When You Run kubectl cordon?

1️⃣ **Sets the node to `SchedulingDisabled` mode:**

```
kubectl get nodes
```

```
NAME STATUS ROLES AGE VERSION
master-node Ready,SchedulingDisabled master 300d v1.29.0
```

2️⃣ **New Pods Are NOT Scheduled on This Node:**

- The **scheduler does NOT assign new workloads** to a cordoned node.
- **Existing workloads remain running.**

Why Do We Need kubectl cordon?

1️⃣ **Prevents New Workloads on a Node Before Upgrading or Decommissioning**

- Ensures no new workloads get assigned **before draining the node**.

2️⃣ **Keeps the Node Running While Preparing for Maintenance**

- You can **monitor existing workloads** before draining them.

One more imp concept regarding cluster upgrades - PDB

What is PDB (Pod Disruption Budget) in Kubernetes?

✅ Pod Disruption Budget (PDB) is a Kubernetes feature that prevents too many replicas of an application from being unavailable during voluntary disruptions (like node drain, updates, or maintenance).

In simple terms:

- It sets limits on how many pods can be disrupted at once in case of node maintenance or scaling.
- It ensures high availability by preventing Kubernetes from evicting too many pods at the same time.
- It is only applicable to voluntary disruptions, like node drain or cluster upgrades (not for pod crashes or failures).

Why Do We Need PDB?

Without PDB:

- If a node goes for maintenance, Kubernetes may evict all replicas of an application, making it unavailable.
- Critical services (e.g., databases, web apps) can go down due to excessive pod evictions.

With PDB:

- It ensures a minimum number of pods are always running.
- Helps maintain application uptime during upgrades, auto-scaling, and voluntary node terminations.

How to Define a PDB in Kubernetes?

A Pod Disruption Budget is defined as a YAML manifest and applied to Kubernetes.

Example: PDB for a Web App

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
name: web-app-pdb
spec:
minAvailable: 2 # At least 2 pods should always be running
selector:
matchLabels:
app: web-app # Targets the pods with this label
```

✅ What this does:

- Prevents more than 1 pod from being evicted at a time.
- Ensures that at least 2 replicas are always running.

PDB Parameters Explained

You can set PDB using either `minAvailable` or `maxUnavailable`:

Field	Description
`minAvailable`	Minimum number of pods that should always be running. Example: `minAvailable: 2` means at least 2 pods should stay running.
`maxUnavailable`	Maximum number of pods that can be unavailable at a time. Example: `maxUnavailable: 1` means at most 1 pod can be evicted.

Example: Using `maxUnavailable` Instead

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
name: db-pdb
spec:
```

```
maxUnavailable: 1 # At most 1 pod can be unavailable
selector:
matchLabels:
app: database
```

✅ What this does:

- Ensures that at least 1 database pod is always running.
- Prevents all database pods from being evicted simultaneously.

When is PDB Used in Kubernetes?

🔹 During Node Maintenance (`kubectl drain`)

- Without PDB, all pods on the node might be evicted at once, causing downtime.
- With PDB, Kubernetes evicts only allowed pods, keeping the application running.

🔹 During Cluster Upgrades (Rolling Updates)

- Ensures Kubernetes does not terminate too many application replicas at once.

🔹 In Auto-scaling Scenarios

- Prevents critical pods from being scaled down to zero replicas.

How to Apply PDB in Kubernetes?

```
kubectl apply -f pdb.yaml
kubectl get pdb
```

✅ **Output Example:**

```
NAME MIN AVAILABLE MAX UNAVAILABLE ALLOWED DISRUPTIONS AGE
web-app-pdb 2 N/A 1 5m
```

Limitations of PDB

❌ Does NOT prevent pod failures → PDB only works for voluntary disruptions (not crashes or OOM issues). ❌ Works only with ReplicaSets, Deployments, StatefulSets → Not for single-pod apps.

Summary

Feature	Explanation
Purpose	Ensures a minimum number of pods remain available during voluntary disruptions.
Used in	Node drains, rolling updates, autoscaling scenarios.
Key Parameters	`minAvailable` (Minimum pods that must stay) OR `maxUnavailable` (Maximum pods that can be removed).
Not for	Pod crashes, node failures, non-replicated workloads.

Final Verdict: ✅ PDB is crucial for production Kubernetes clusters to ensure high availability and prevent accidental downtime during upgrades and maintenance!

Why Do We Need PDB When Upgrading a Kubernetes Cluster?

When you drain the first worker node during a Kubernetes cluster upgrade, the pods running on that node are rescheduled onto the remaining worker nodes (if resources are available). However, Pod Disruption Budget (PDB) is still important because it controls how many pods can be evicted at a time, ensuring high availability of applications.

1️⃣ Does PDB Apply to the Node Being Upgraded or the Whole Cluster?

✅ PDB applies at the cluster level, not just the node being drained. ✅ It prevents too many critical pods from being evicted at the same time across the entire cluster.

💡 Example Scenario (Without PDB):

- Suppose your application has 3 replicas spread across 5 worker nodes.
- If you drain 3 nodes one after another, all replicas might get evicted at the same time.

- This causes temporary downtime for the application.

💡 With PDB (e.g., `minAvailable: 2`)

- Kubernetes ensures at least 2 replicas are always running before evicting another pod.
- It stalls the node drain process until new pods are scheduled on available nodes.

2️⃣ Why Is PDB Needed If Pods Are Automatically Rescheduled?

🔹 Auto-rescheduling ≠ Instant Readiness

- Even though pods are rescheduled, they take time to restart.
- Example: If your application needs 30 seconds to initialize, it could cause downtime if all pods are evicted together.

🔹 What If All Nodes Are Upgraded at the Same Time?

- Without PDB, Kubernetes might evict too many replicas of a critical service at the same time.
- For example: If you upgrade all 5 nodes in parallel, there is no guarantee that at least 1 pod is running at all times.

🔹 Helps in StatefulSets and Databases

- Stateful applications (MySQL, PostgreSQL, Kafka) need careful eviction control to avoid split-brain scenarios.
- If all database replicas are evicted at once, it can lead to data corruption or loss of quorum.

Why Haven't You Come Across PDB in Production Cluster Upgrades?

1. Your Application Might Not Have a PDB Set Up
   - If your cluster didn't have PDB defined, Kubernetes drained nodes without restriction.
2. Cluster Had Enough Worker Nodes
   - If your cluster had many nodes, pods were rescheduled without noticeable downtime.
3. The Application Had Auto-Scaling
   - If your cluster used horizontal pod autoscaling (HPA), it likely started new pods before eviction.
4. Your Deployment Type Didn't Need PDB
   - PDB is more important for databases, stateful workloads, and critical services.
   - If your app was stateless (e.g., a microservice without persistent storage), downtime was not noticeable.

Final Answer: When Do You Need PDB in Cluster Upgrades?

Scenario	Do You Need PDB?
Stateless Web Apps	❌ Not mandatory
Database (PostgreSQL, MySQL)	✅ Yes, prevents quorum loss
StatefulSets (Kafka, RabbitMQ)	✅ Yes, avoids disruption
CI/CD Pipelines, Logging	❌ Not required
Large-Scale Microservices	✅ Recommended for high availability

If your production upgrade worked fine without PDB, it's likely because: 1️⃣ You had enough worker nodes. 2️⃣ The application was stateless and could tolerate restarts. 3️⃣ Kubernetes rescheduled pods fast enough without downtime.